ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID



Clasificadores supervidados y regresion

Luis Garmendia Salvador
Departamento de Ingeniería del Software e Inteligencia Artificial
Facultad de Informática, UCM

2

Clasificadores supervidados y regresion

- Regresión lineal, multidimensional, polinomial...
- Naive Bayes
- Maquina de soporte de vectores (SVM)
- Árboles de decisión y Random Forest
- Redes Neuronales



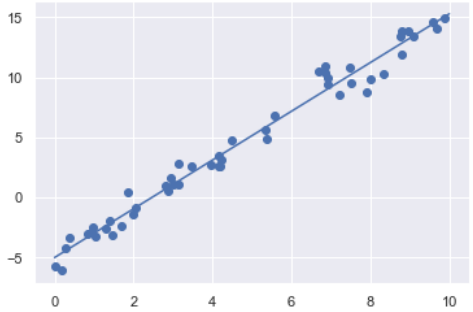
Regresión Lineal

In [153]:
```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)

model.fit(x[:, np.newaxis], y)

xfit = np.linspace(0, 10, 1000)
yfit = model.predict(xfit[:, np.newaxis])

plt.scatter(x, y)
plt.plot(xfit, yfit);
```



Luís Garmendia

Regresión Lineal

$$y = Ax + B$$

A es la pendiente

B es el término independiente

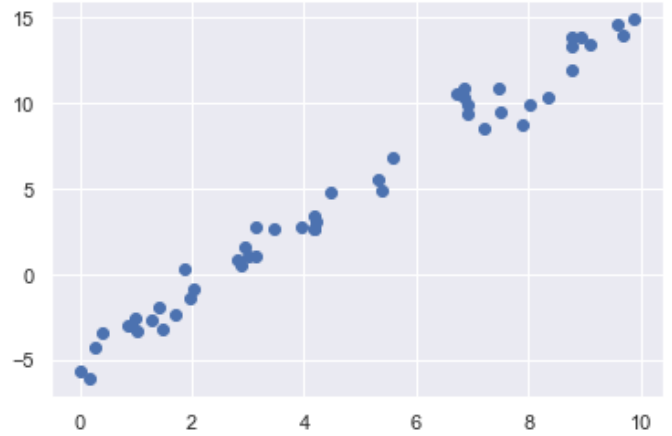
Regresión Lineal

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
import numpy as np
```

Creación de datos:

```
rng = np.random.RandomState(1)
x = 10 * rng.rand(50)
y = 2 * x - 5 + rng.randn(50)
plt.scatter(x, y);
plt.show()
```

```
rng = np.random.RandomState(1)
x = 10 * rng.rand(50)
y = 2 * x - 5 + rng.randn(50)
plt.scatter(x, y);
```



Regresión Lineal

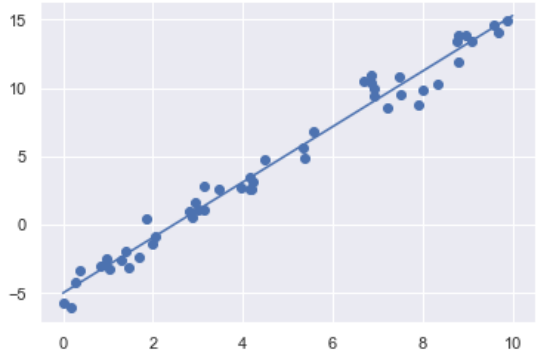
```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)
model.fit(x[:, np.newaxis], y)
xfit = np.linspace(0, 10, 1000)
yfit = model.predict(xfit[:, np.newaxis])
plt.scatter(x, y)
plt.plot(xfit, yfit);
```

```
In [153]: from sklearn.linear_model import LinearRegression
          model = LinearRegression(fit_intercept=True)

          model.fit(x[:, np.newaxis], y)

          xfit = np.linspace(0, 10, 1000)
          yfit = model.predict(xfit[:, np.newaxis])

          plt.scatter(x, y)
          plt.plot(xfit, yfit);
```



Regresión Lineal
slope e intercept (A y B)

```
print("Model slope: ", model.coef_[0])
print("Model intercept:", model.intercept_)
```

```
Model slope: 2.02720881036
Model intercept: -4.99857708555
```

Regresión Lineal Multidimensional

$y=a_0+a_1x_1+a_2x_2+\cdots$

```
rng = np.random.RandomState(1)
X = 10 * rng.rand(100, 3)
y = 0.5 + np.dot(X, [1.5, -2., 1.])
model.fit(X, y)
print(model.intercept_)
print(model.coef_)
```

```
0.5
[ 1.5 -2. 1. ]
```

Regresión Polinomial,
PolynomialFeatures transformer

$y=a_0+a_1x+a_2x^2+a_3x^3+ \ldots$

from sklearn.preprocessing import PolynomialFeatures
x = np.array([2, 3, 4])
poly = PolynomialFeatures(3, include_bias=False)
poly.fit_transform(x[:, None])

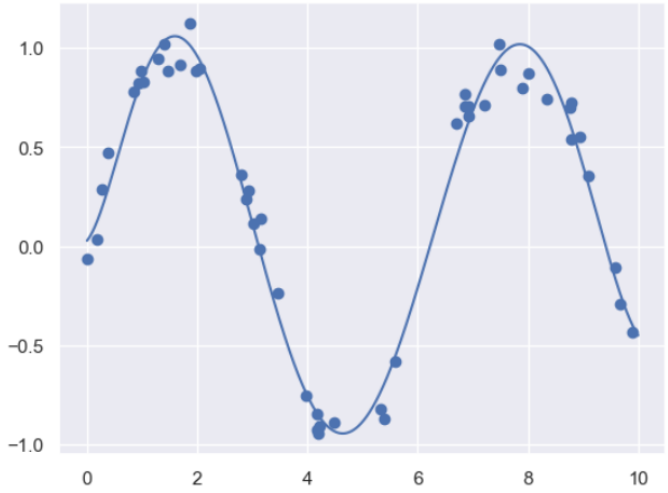
```
from sklearn.preprocessing import PolynomialFeatures
x = np.array([2, 3, 4])
poly = PolynomialFeatures(3, include_bias=False)
poly.fit_transform(x[:, None])
```

```
array([[ 2.,  4.,  8.],
       [ 3.,  9., 27.],
       [ 4., 16., 64.]])
```

Regresión Polinomial,

PolynomialFeatures transformer

```
from sklearn.pipeline import make_pipeline
poly_model = make_pipeline(PolynomialFeatures(7), LinearRegression())
rng = np.random.RandomState(1)
x = 10 * rng.rand(50)
y = np.sin(x) + 0.1 * rng.randn(50)
poly_model.fit(x[:, np.newaxis], y)
yfit = poly_model.predict(xfit[:, np.newaxis])
plt.scatter(x, y)
plt.plot(xfit, yfit);
plt.show()
```



Regresión Polinomial,
PolynomialFeatures transformer

```
from sklearn.pipeline import make_pipeline
poly_model = make_pipeline(PolynomialFeatures(7), LinearRegression())
rng = np.random.RandomState(1)
x = 10 * rng.rand(50)
y = np.sin(x) + 0.1 * rng.randn(50)
poly_model.fit(x[:, np.newaxis], y)
yfit = poly_model.predict(xfit[:, np.newaxis])
plt.scatter(x, y)
plt.plot(xfit, yfit);
plt.show()
```

Regresión Polinomial,
PolynomialFeatures transformer

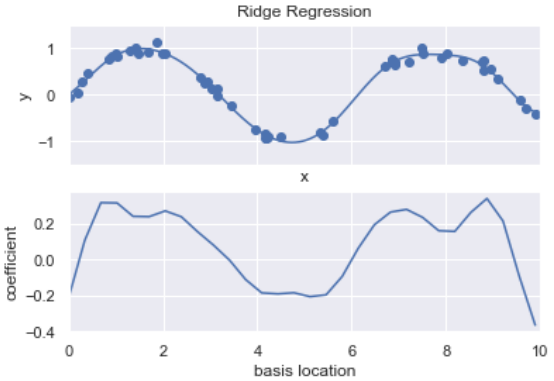
```
from sklearn.pipeline import make_pipeline
poly_model = make_pipeline(PolynomialFeatures(7), LinearRegression())
rng = np.random.RandomState(1)
x = 10 * rng.rand(50)
y = np.sin(x) + 0.1 * rng.randn(50)
poly_model.fit(x[:, np.newaxis], y)
yfit = poly_model.predict(xfit[:, np.newaxis])
plt.scatter(x, y)
plt.plot(xfit, yfit);
plt.show()
```

Ridge regression

```
from sklearn.linear_model import Ridge
model = make_pipeline(GaussianFeatures(30), Ridge(alpha=0.1))
basis_plot(model, title='Ridge Regression')
plt.show()
```

$$P = \alpha \sum_{n=1}^{N} \theta_n^2$$

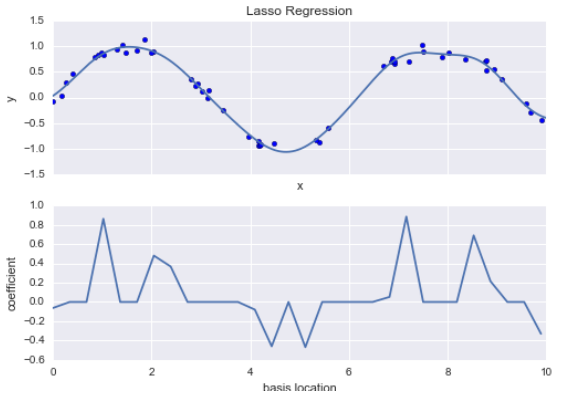
```
In [163]: from sklearn.linear_model import Ridge
          model = make_pipeline(GaussianFeatures(30), Ridge(alpha=0.1))
          basis_plot(model, title='Ridge Regression')
```



Lasso regression

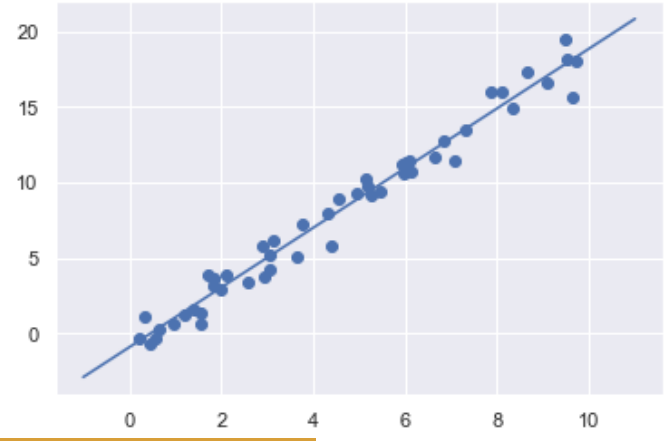
$$P = \alpha \sum_{n=1}^{N} |\theta_n|$$

```
from sklearn.linear_model import Lasso
model = make_pipeline(GaussianFeatures(30), Lasso(alpha=0.001))
basis_plot(model, title='Lasso Regression')
```



from sklearn.linear_model import Lasso
model = make_pipeline(GaussianFeatures(30),
basis_plot(model, title='Lasso Regression')
plt.show()

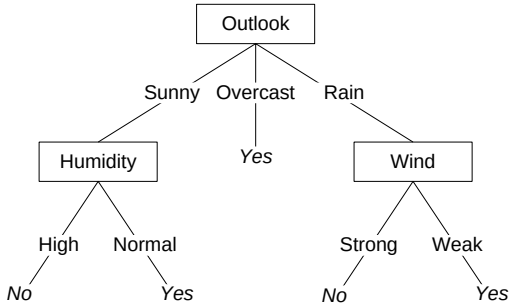
Clasificación con Naive Bayes (supervisado)



Luís Garmendia

Naïve Bayes

El modelo Naive Bayes es rápido y simple, válido para muchas dimensiones y sin parámetros.



Day	*Outlook*	*Temperature*	*Humidity*	*Wind*	*PlayTennis*
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

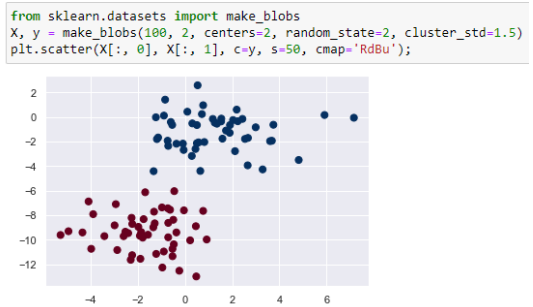
$$P(L \mid \text{features}) = \frac{P(\text{features} \mid L)P(L)}{P(\text{features})}$$

Gaussian Naïve Bayes

Generación de datos:

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()

from sklearn.datasets import make_blobs
X, y = make_blobs(100, 2, centers=2, random_state=2, cluster_std=1.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu');
```



Gaussian Naïve Bayes

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X, y);
```

Gaussian Naïve Bayes

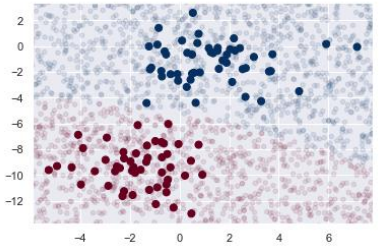
```
rng = np.random.RandomState(0)
Xnew = [-6, -14] + [14, 18] * rng.rand(2000, 2)
ynew = model.predict(Xnew)
```

```
In [141]: ynew
Out[141]: array([1, 1, 1, ..., 0, 1, 1])
```

No o get an idea c

```
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu')
lim = plt.axis()
plt.scatter(Xnew[:, 0], Xnew[:, 1], c=ynew, s=20, cmap='RdBu', alpha=0.1)
plt.axis(lim);
```

```
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu')
lim = plt.axis()
plt.scatter(Xnew[:, 0], Xnew[:, 1], c=ynew, s=20, cmap='RdBu', alpha=0.1)
plt.axis(lim);
```



Gaussian Naïve Bayes predict_proba

Podemos pedir la probabilidad calpculada para cada y. Pensar en casos de medicina para diagnósticos de cancer…

```
In [135]: yprob = model.predict_proba(Xnew)
          yprob[-8:].round(2)

Out[135]: array([[0.89, 0.11],
                 [1.  , 0.  ],
                 [1.  , 0.  ],
                 [1.  , 0.  ],
                 [1.  , 0.  ],
                 [1.  , 0.  ],
                 [0.  , 1.  ],
                 [0.15, 0.85]])
```

Clasificación de Textos

```
from sklearn.datasets import fetch_20newsgroups
data = fetch_20newsgroups()
data.target_names
```

```
['alt.atheism',
'comp.graphics',
'comp.os.ms-windows.misc',
'comp.sys.ibm.pc.hardware',
'comp.sys.mac.hardware',
'comp.windows.x', …,
'talk.politics.misc', 'talk.religion.misc']
```

Clasificación de Textos

Por simplicidad reducimos las categorias

```
categories = ['talk.religion.misc', 'soc.religion.christian', 'sci.space', 'comp.graphics']
train = fetch_20newsgroups(subset='train', categories=categories)
test = fetch_20newsgroups(subset='test', categories=categories)
```

Ejemplo de datos:

```
print(train.data[5])
```

```
In [144]: print(train.data[5])

From: dmcgee@uluhe.soest.hawaii.edu (Don McGee)
Subject: Federal Hearing
Originator: dmcgee@uluhe
Organization: School of Ocean and Earth Science and Technology
Distribution: usa
Lines: 10


Fact or rumor....?  Madalyn Murray O'Hare an atheist who eliminated the
use of the bible reading and prayer in public schools 15 years ago is now
going to appear before the FCC with a petition to stop the reading of the
Gospel on the airways of America.  And she is also campaigning to remove
Christmas programs, songs, etc from the public schools.  If it is true
then mail to Federal Communications Commission 1919 H Street Washington DC
20054 expressing your opposition to her request.  Reference Petition number
```

Clasificación de Textos: pipeline de vectorización + clasificación

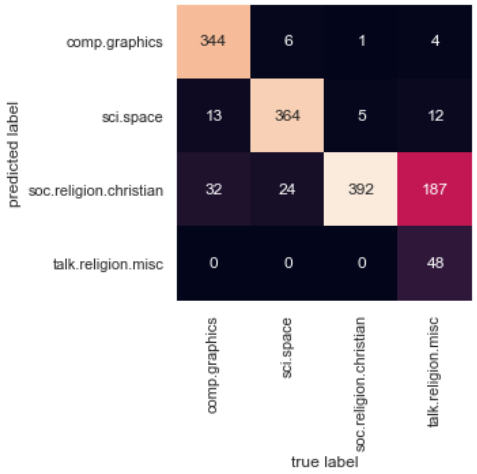
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
model = make_pipeline(TfidfVectorizer(), MultinomialNB())
```

Classifying Text

```
model.fit(train.data, train.target)
labels = model.predict(test.data)
```

Clasificación de Textos
Matriz de confusion

```
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(test.target, labels)
sns.heatmap(mat.T, square=True, annot=True, fmt='d', cbar=False,
            xticklabels=train.target_names, yticklabels=train.target_names)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



Clasificación de Textos
Función para clasificar textos

```
In [148]: def predict_category(s, train=train, model=model):
              pred = model.predict([s])
              return train.target_names[pred[0]]

In [149]: predict_category('sending a payload to the ISS')

Out[149]: 'sci.space'

In [150]: predict_category('discussing islam vs atheism')

Out[150]: 'soc.religion.christian'

In [151]: predict_category('determining the screen resolution')

Out[151]: 'comp.graphics'
```

Clasificación de Textos
Función para clasificar textos

```
def predict_category(s, train=train, model=model):
        pred = model.predict([s])
        return train.target_names[pred[0]]
predict_category('sending a payload to the ISS')
predict_category('discussing islam vs atheism')
predict_category('determining the screen resolution')
```

Cuando usar Naïve Bayes

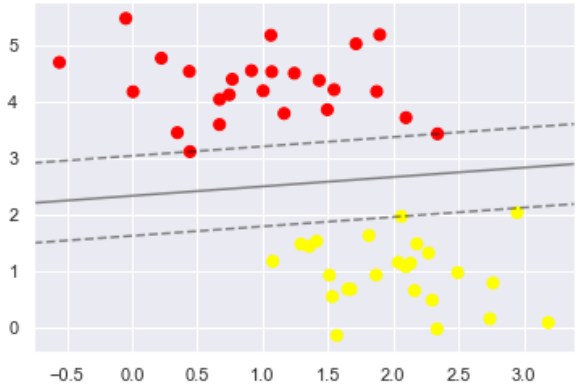
Es un clasificador sencillo, poco potente, sus ventajas son:

- Muy rápido en entrenamiento y predicción
- Predicción probabilística
- Fácil de interpretar
- Sin parámetros
- Funciona bien en datos separados



Maquina de Soporte de Vectores
Support Vector Machines (SVM)

```
In [171]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
          plot_svc_decision_function(model);
```



Luís Garmendia

Support vector machines (SVMs)

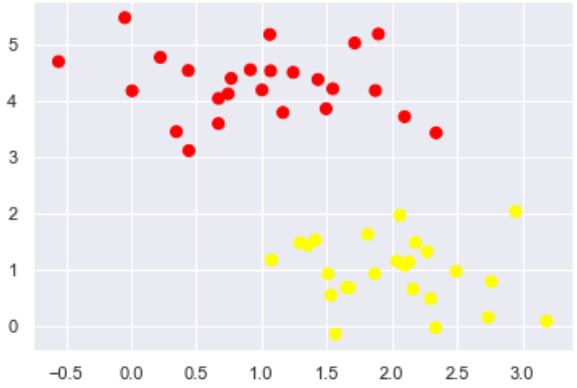
Son una clase de algoritmos de clasificación y regresión supervisados

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
# use seaborn plotting defaults
import seaborn as sns; sns.set()
```

Support vector machines (SVMs)

Ejemplo de datos:

```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=50, centers=2,
                  random_state=0, cluster_std=0.60)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn');
```

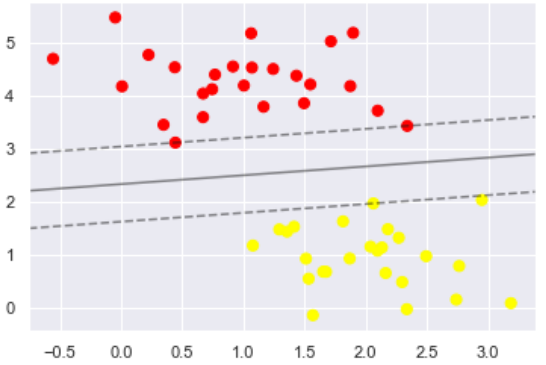


Support vector machines (SVMs)

El clasificador lineal discriminativo intenta dibujar una línea de anchura C que separe los datos

```
from sklearn.svm import SVC # "Support vector classifier"
model = SVC(kernel='linear', C=1E10)
model.fit(X, y)
```

```
In [171]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
          plot_svc_decision_function(model);
```



Support vector machines (SVMs)

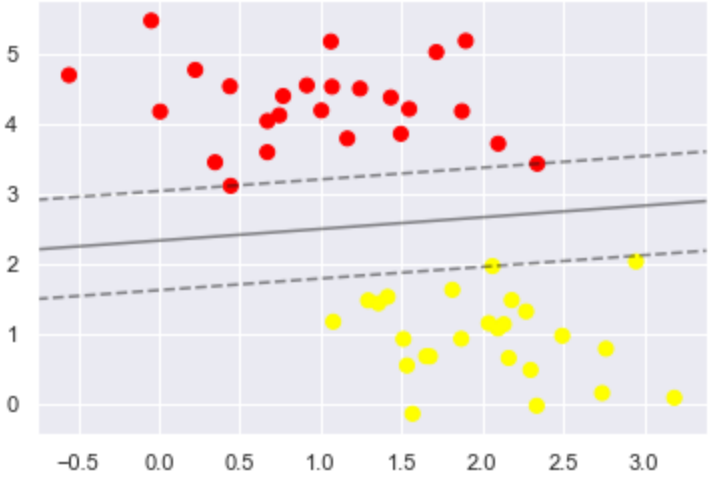
To better visualize what's happening here, let's create a quick convenience function that will plot SVM decision boundaries for us:

```
def plot_svc_decision_function(model, ax=None, plot_support=True):
    """Plot the decision function for a 2D SVC"""
    if ax is None:
        ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
    # create grid to evaluate model
    x = np.linspace(xlim[0], xlim[1], 30)
    y = np.linspace(ylim[0], ylim[1], 30)
    Y, X = np.meshgrid(y, x)
    xy = np.vstack([X.ravel(), Y.ravel()]).T
    P = model.decision_function(xy).reshape(X.shape)
    # plot decision boundary and margins
    ax.contour(X, Y, P, colors='k',
               levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])
    # plot support vectors
    if plot_support:
        ax.scatter(model.support_vectors_[:, 0],
                   model.support_vectors_[:, 1],
                   s=300, linewidth=1, facecolors='none');
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
```

Support vector machines (SVMs)

```
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(model);
```

```
In [171]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
          plot_svc_decision_function(model);
```



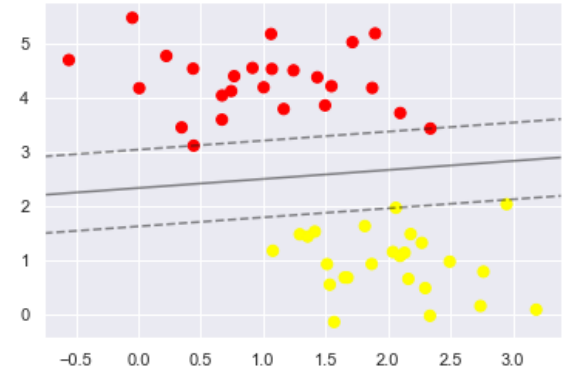
Support vector machines
support_vectors_

La línea divisoria que separa los datos se calcula solo por unos pocos puntos llamados vectores de soporte.

model.support_vectors_

array([[0.44359863, 3.11530945],
[2.33812285, 3.43116792],
[2.06156753, 1.96918596]])

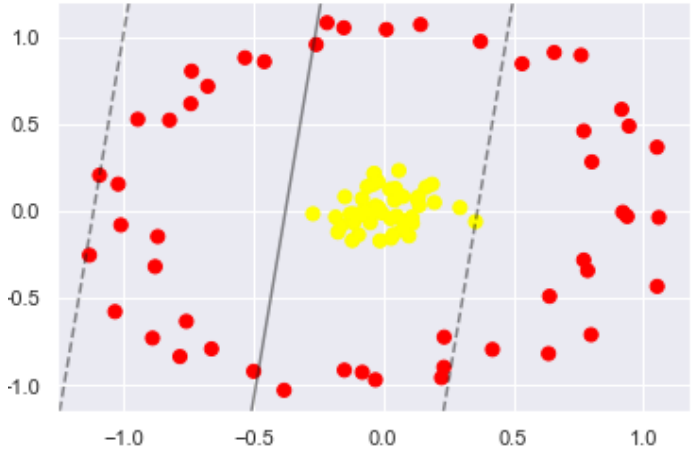
```
In [171]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
          plot_svc_decision_function(model);
```



Kernel SVM

Los kernel sirven para datos no linalmente separables

```
from sklearn.datasets.samples_generator import make_circles
X, y = make_circles(100, factor=.1, noise=.1)
clf = SVC(kernel='linear').fit(X, y)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(clf, plot_support=False);
```



Kernel SVM RBF

radial basis function:

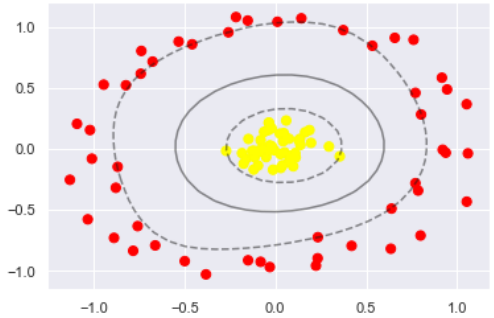
```
r = np.exp(-(X ** 2).sum(1))

clf =SVC(kernel='rbf', C=1E6)
clf.fit(X, y)
```

```
In [179]: clf = SVC(kernel='rbf', C=1E6)
          clf.fit(X, y)

Out[179]: SVC(C=1000000.0, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,
              decision_function_shape='ovr', degree=3, gamma='scale', kernel='rbf',
              max_iter=-1, probability=False, random_state=None, shrinking=True,
              tol=0.001, verbose=False)

In [180]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
          plot_svc_decision_function(clf)
          plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
                      s=300, lw=1, facecolors='none');
```



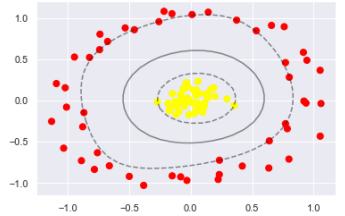
Kernel SVM
RBF

```
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(clf)
plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1], s=300, lw=1,
facecolors='none');
```

```
In [179]: clf = SVC(kernel='rbf', C=1E6)
          clf.fit(X, y)

Out[179]: SVC(C=1000000.0, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,
              decision_function_shape='ovr', degree=3, gamma='scale', kernel='rbf',
              max_iter=-1, probability=False, random_state=None, shrinking=True,
              tol=0.001, verbose=False)

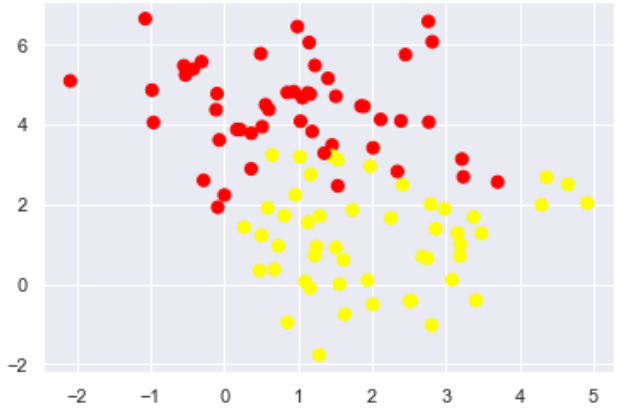
In [180]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
          plot_svc_decision_function(clf)
          plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
                      s=300, lw=1, facecolors='none');
```



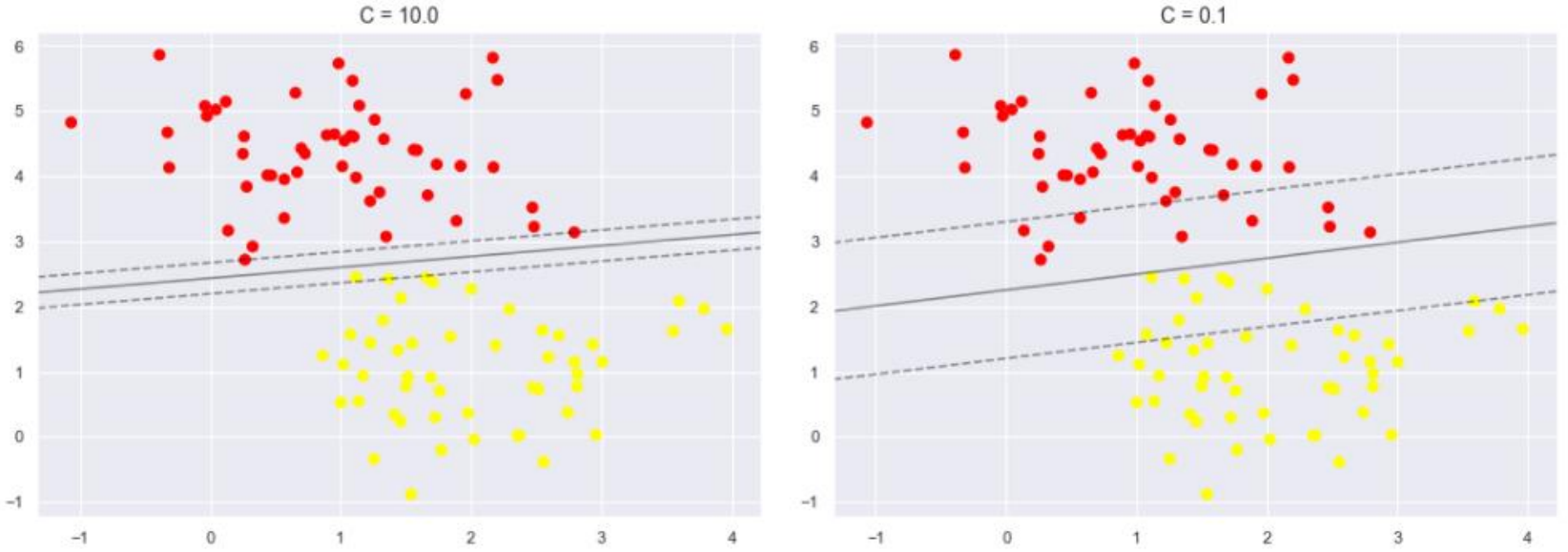
Solapamiento SVM –overlap-

X, y = make_blobs(n_samples=100, centers=2, random_state=0, cluster_std=1.2)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn');

```
In [181]: X, y = make_blobs(n_samples=100, centers=2,
                            random_state=0, cluster_std=1.2)
          plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn');
```



SVM overlap
C parameter

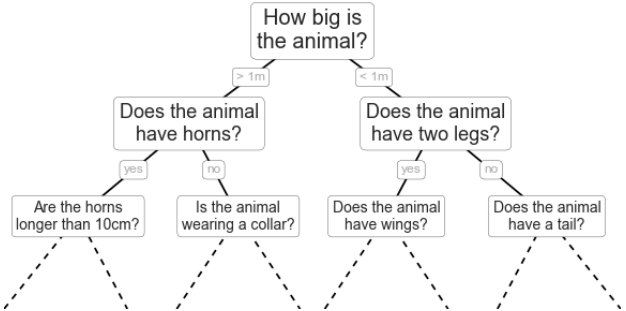


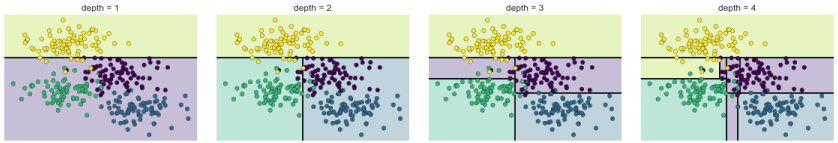
Ventajas del SVM

- Al depender de unos pocos vectores el cálculo require poca memoria y funciona bien con datos de alta dimensión
- Una vez entrenado el modelo, la predicción es muy rápida.
- Integración con métodos de kernel para adaptarse a los datos
- Parametro C debe ser ajustado y validado
- No tiene interpretación probabilística



Decision Trees
Random Forest







Luís Garmendia

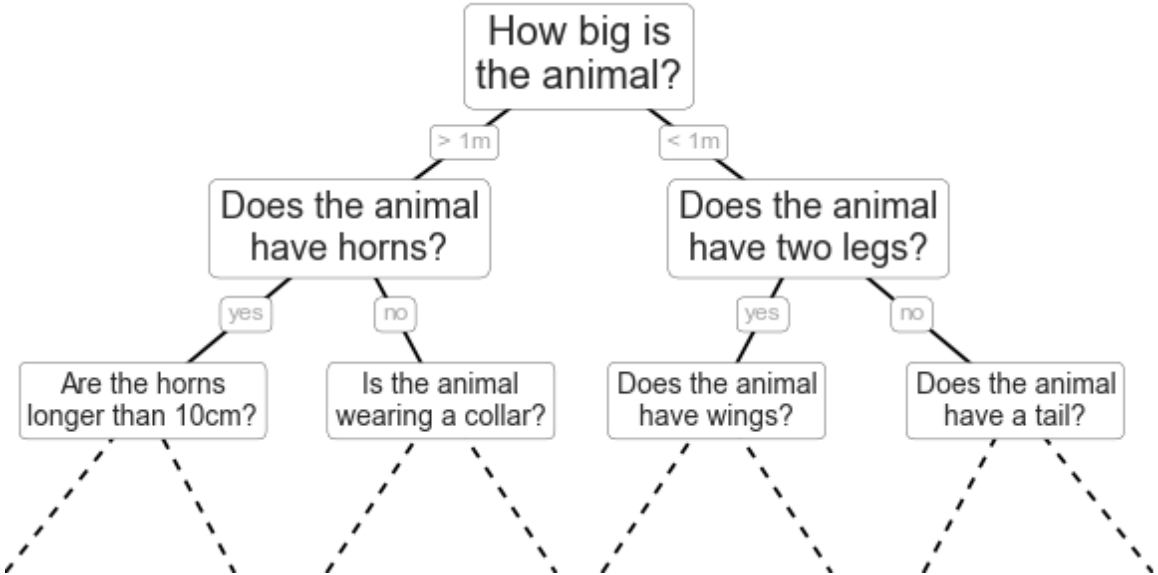
Random Forest

Random forests es un método que aggrega los resultados de muchos a´rboles de decision, tomando la moyoría de votos.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
```

Decision Tree

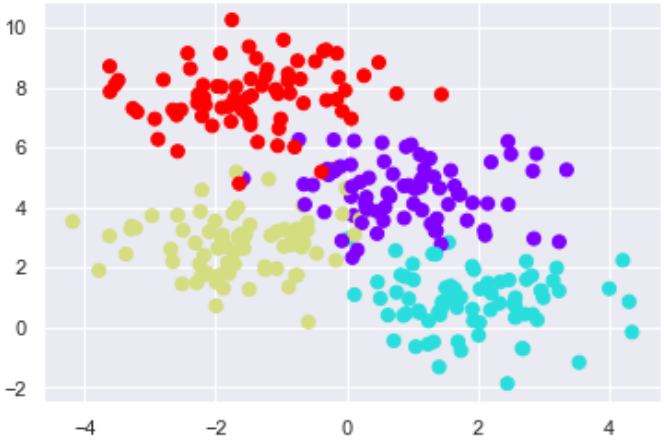
Se construye preguntando a los datos, separandolos en dos en cada pregunta



Decision Tree

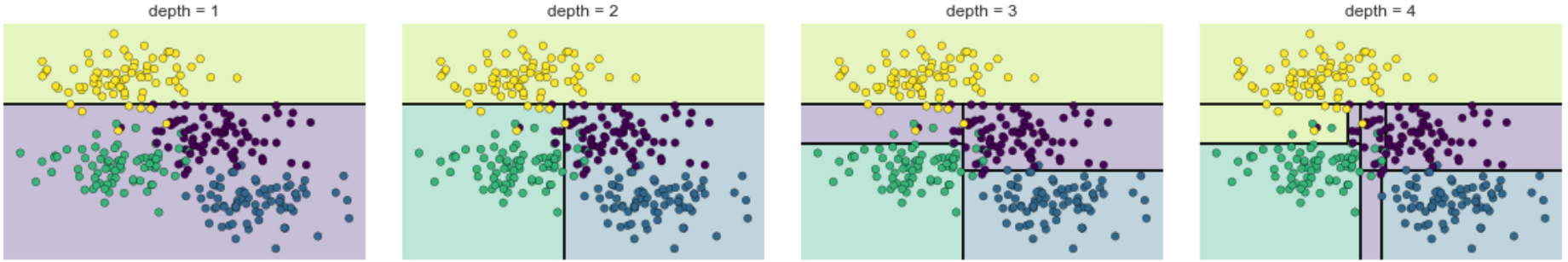
Datos bidimensionales con 4 categorías (labels):

```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=300, centers=4, random_state=0, cluster_std=1.0)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='rainbow');
```



Decision Tree

Niveles del árbol de decision:



Decision Tree

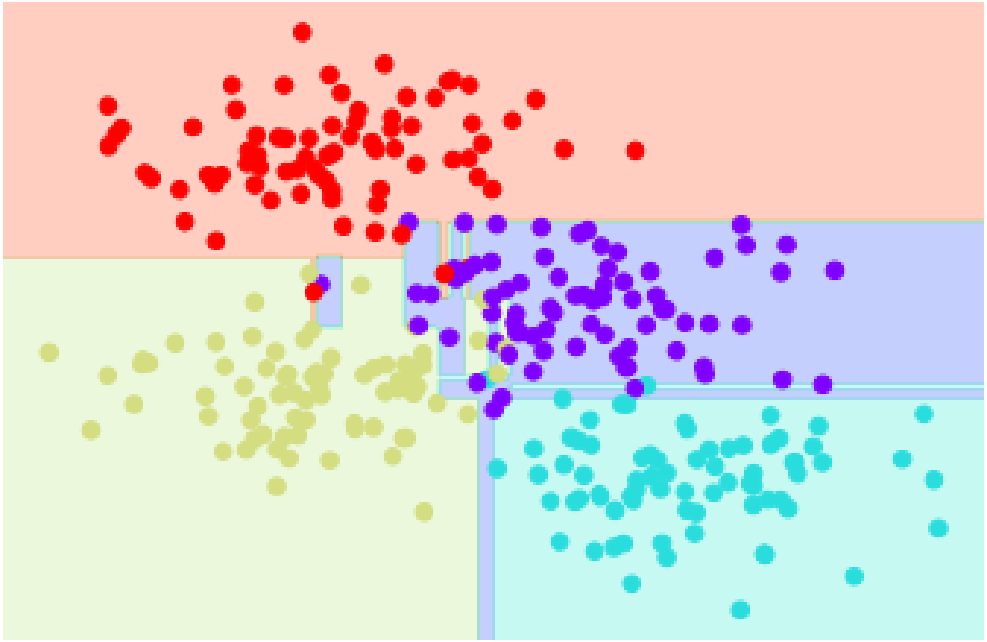
```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier().fit(X, y)
```

Decision Tree

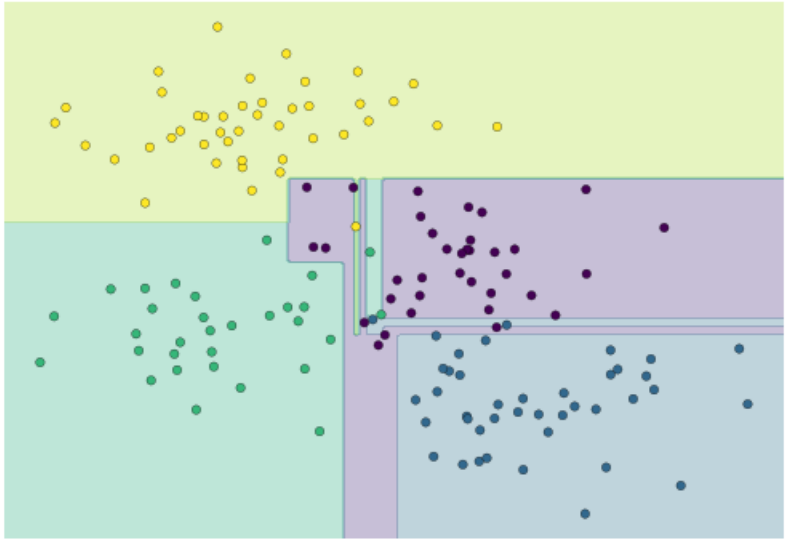
```
def visualize_classifier(model, X, y, ax=None, cmap='rainbow'):
    ax = ax or plt.gca()
    ax.scatter(X[:, 0], X[:, 1], c=y, s=30, cmap=cmap, clim=(y.min(), y.max()), zorder=3)
    ax.axis('tight')
    ax.axis('off')
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
    # fit the estimator
    model.fit(X, y)
    xx, yy = np.meshgrid(np.linspace(*xlim, num=200), np.linspace(*ylim, num=200))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    # Create a color plot with the results
    n_classes = len(np.unique(y))
    contours = ax.contourf(xx, yy, Z, alpha=0.3, levels=np.arange(n_classes + 1) - 0.5, cmap=cmap, clim=(y.min(),
y.max()), zorder=1)
    ax.set(xlim=xlim, ylim=ylim)
```

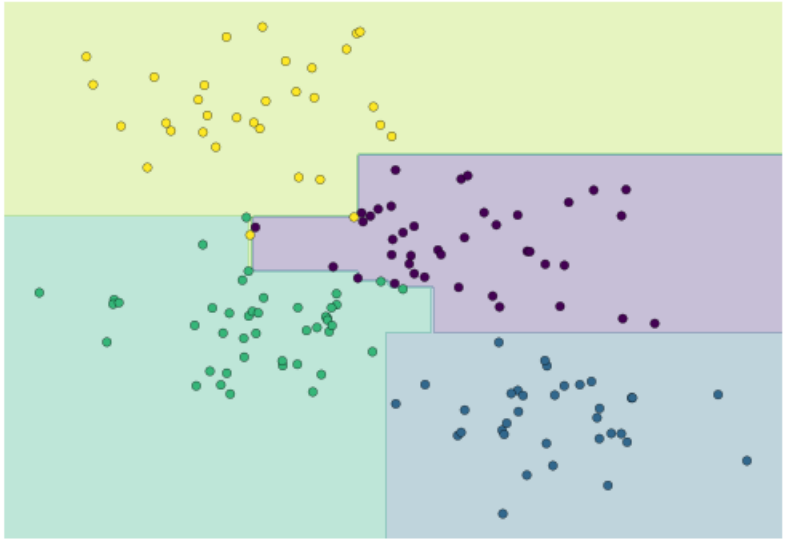
Decision Tree

visualize_classifier(DecisionTreeClassifier(), X, y)



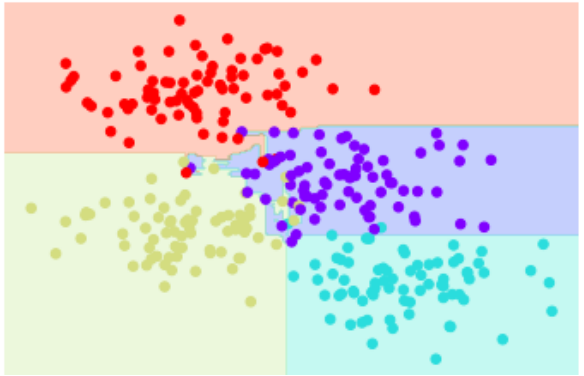
Varios Decision Trees





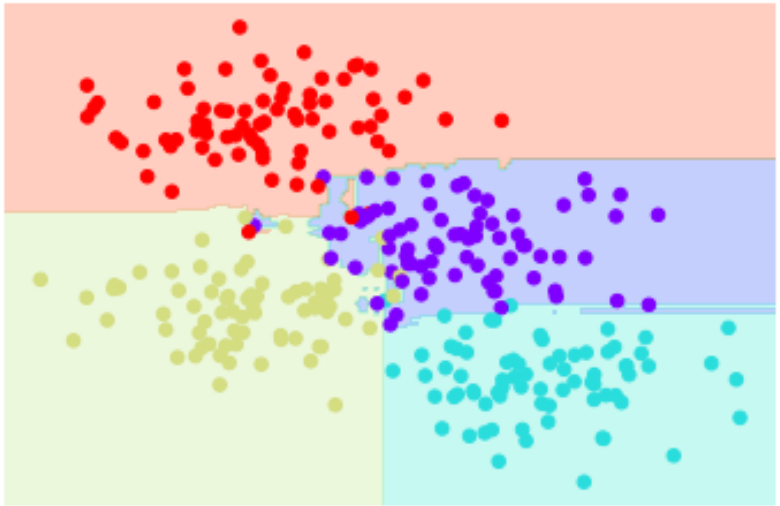
Random Forest

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
tree = DecisionTreeClassifier()
bag = BaggingClassifier(tree, n_estimators=100,
        max_samples=0.8, random_state=1)
bag.fit(X, y)
visualize_classifier(bag, X, y)
```



Random Forest

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=0)
visualize_classifier(model, X, y);
```



Clasificando digitos

```
from sklearn.datasets import load_digits
digits = load_digits()
digits.keys()
```

```
from sklearn.datasets import load_digits
digits = load_digits()
# set up the figure
fig = plt.figure(figsize=(6, 6)) # figure size in inches
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
# plot the digits: each image is 8x8 pixels
for i in range(64):
    ax = fig.add_subplot(8, 8, i + 1, xticks=[], yticks=[])
    ax.imshow(digits.images[i], cmap=plt.cm.binary, interpolation='nearest')
    # label the image with the target value
    ax.text(0, 7, str(digits.target[i]))
```

Clasificando digitos

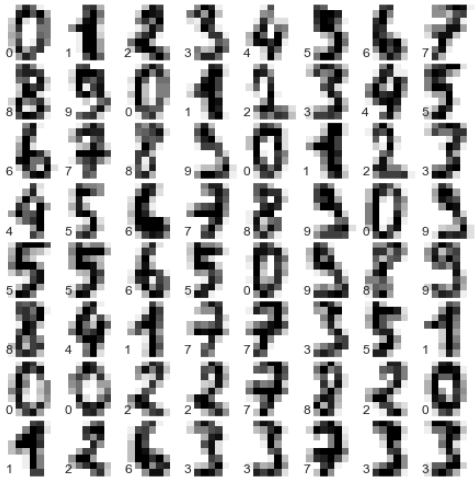
In [206]:
```
from sklearn.datasets import load_digits
digits = load_digits()
digits.keys()
```

Out[206]:
```
dict_keys(['data', 'target', 'target_names', 'images', 'DESCR'])
```

In [207]:
```
# set up the figure
fig = plt.figure(figsize=(6, 6))  # figure size in inches
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)

# plot the digits: each image is 8x8 pixels
for i in range(64):
    ax = fig.add_subplot(8, 8, i + 1, xticks=[], yticks=[])
    ax.imshow(digits.images[i], cmap=plt.cm.binary, interpolation='nearest')

    # label the image with the target value
    ax.text(0, 7, str(digits.target[i]))
```



Clasificando digitos

```
from sklearn.model_selection import train_test_split
Xtrain, Xtest, ytrain, ytest = train_test_split(digits.data, digits.target,
random_state=0)
model = RandomForestClassifier(n_estimators=1000)
model.fit(Xtrain, ytrain)
ypred = model.predict(Xtest)
```

Clasificando digitos

```
from sklearn import metrics
print(metrics.classification_report(ypred, ytest))
```

```
              precision    recall  f1-score   support

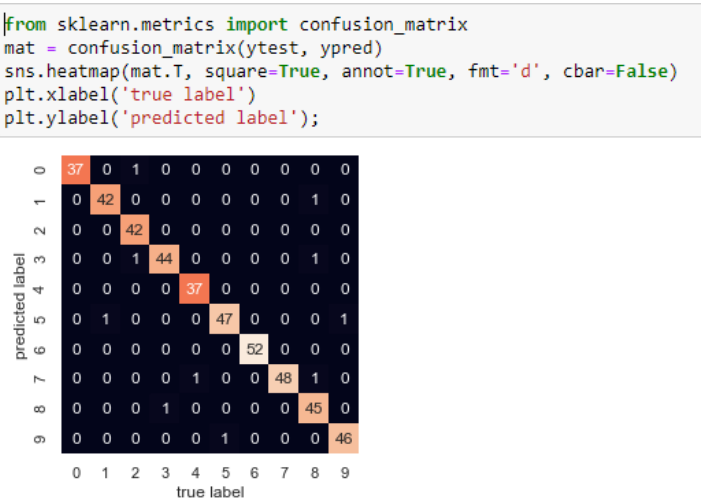
           0       1.00      0.97      0.99        38
           1       0.98      0.98      0.98        43
           2       0.95      1.00      0.98        42
           3       0.98      0.96      0.97        46
           4       0.97      1.00      0.99        37
           5       0.98      0.96      0.97        49
           6       1.00      1.00      1.00        52
           7       1.00      0.96      0.98        50
           8       0.94      0.98      0.96        46
           9       0.98      0.98      0.98        47

    accuracy                           0.98       450
   macro avg       0.98      0.98      0.98       450
weighted avg       0.98      0.98      0.98       450
```

Clasificando digitos

Matriz de confusion:

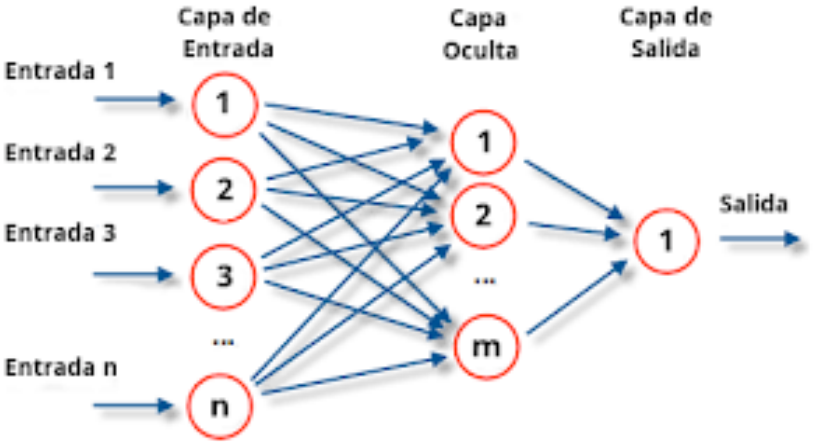
```
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(ytest, ypred)
sns.heatmap(mat.T, square=True, annot=True, fmt='d', cbar=False)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



Ventajas de Random Forest

- Rapido entrenamiento y predicción
- Tareas paralelizables
- Pocos parámetros
- Resultados mal interpetables y mal explicables

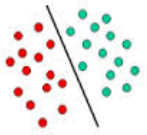
Neuronal Networks



https://scikit-learn.org/stable/modules/neural_networks_supervised.html

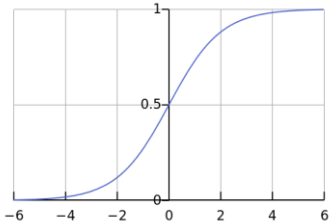
Luís Garmendia

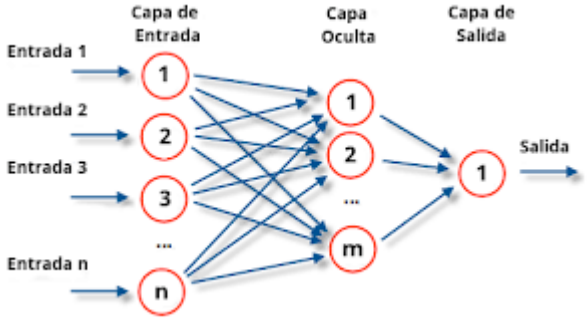
Regresión logística

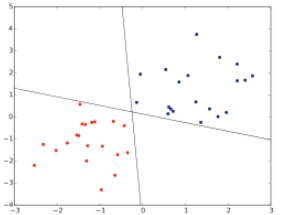


Classification
(LR, SVM…)

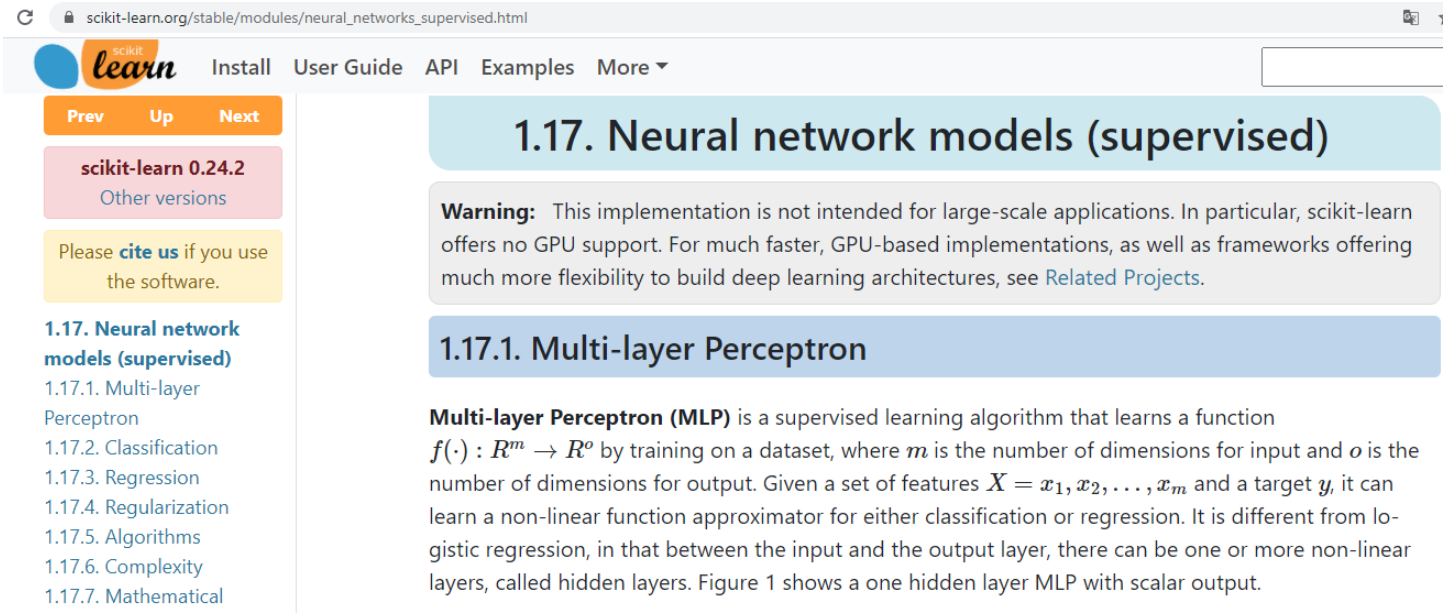
La regresión logística se utiliza para entrenar perceptrones de redes neuronales, entrenandolas para ajustar sus pesos y clasificar





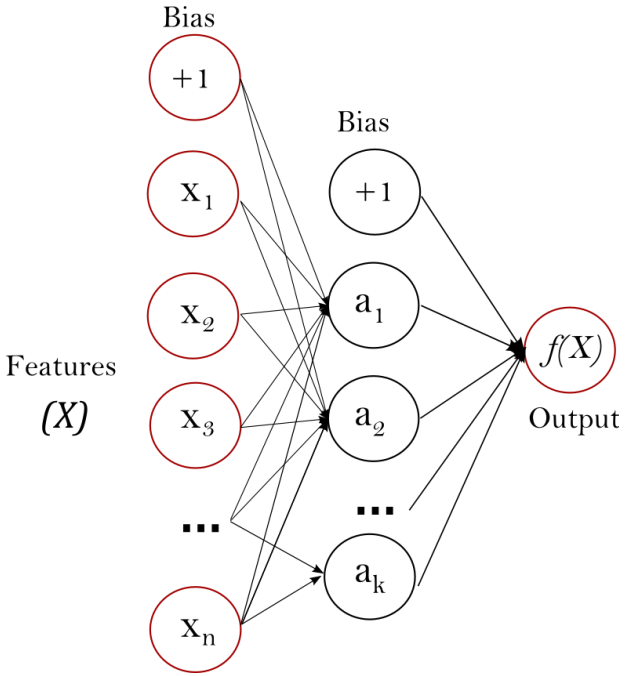


Multi-layer Perceptron



https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Multi-layer Perceptron

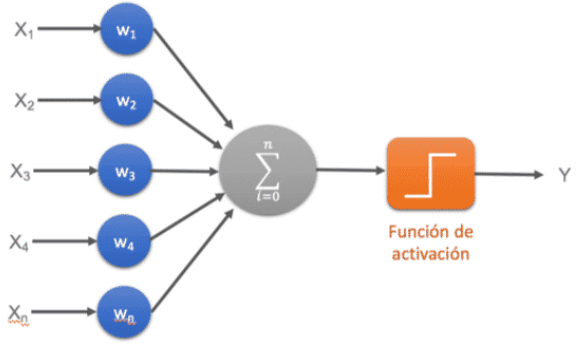


https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Multi-layer Perceptron

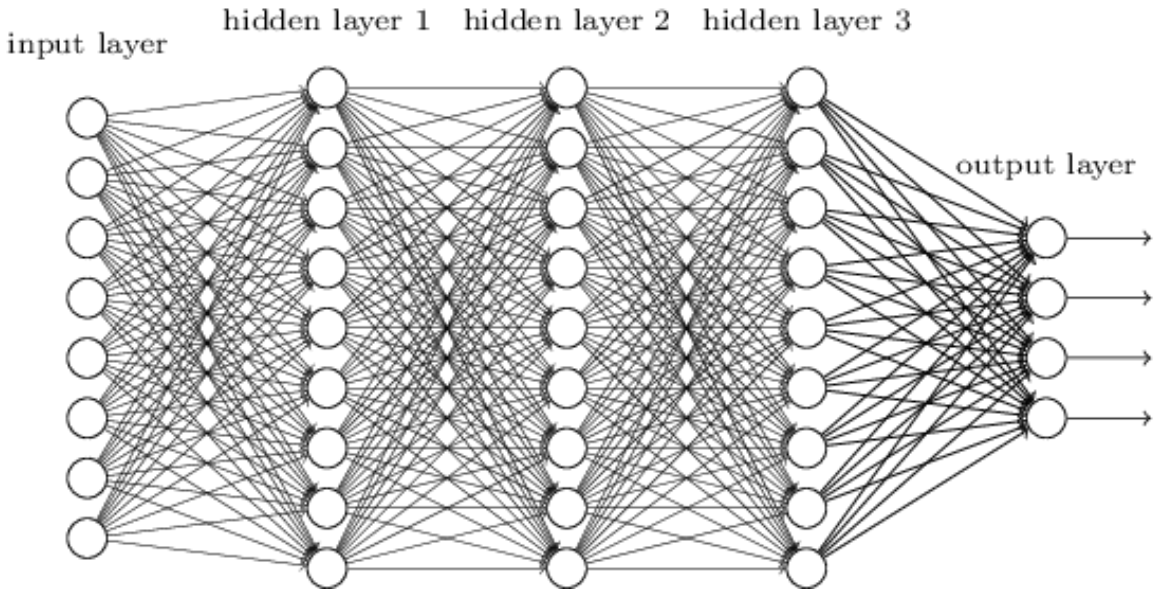
The leftmost layer, known as the input layer, consists of a set of neurons $\{x_i | x_1, x_2, \ldots, x_m\}$ representing the input features. Each neuron in the hidden layer transforms the values from the previous layer with a weighted linear summation $w_1x_1 + w_2x_2 + \ldots + w_mx_m$, followed by a non-linear activation function $g(\cdot) : R \rightarrow R$ - like the hyperbolic tan function. The output layer receives the values from the last hidden layer and transforms them into output values.

The module contains the public attributes `coefs_` and `intercepts_`. `coefs_` is a list of weight matrices, where weight matrix at index i represents the weights between layer i and layer $i + 1$. `intercepts_` is a list of bias vectors, where the vector at index i represents the bias values added to layer $i + 1$.



	Función	Rango	Gráfica
Identidad	$y = x$	$[-\infty, +\infty]$	
Escalón	$y = sign(x)$ $y = H(x)$	$\{-1, +1\}$ $\{0, +1\}$	
Lineal a tramos	$y = \begin{cases} -1, & si\ x < -l \\ x, & si\ +l \le x \le -l \\ +1, & si\ x > +l \end{cases}$	$[-1, +1]$	
Sigmoidea	$y = \frac{1}{1+e^{-x}}$ $y = tgh(x)$	$[0, +1]$ $[-1, +1]$	
Gaussiana	$y = Ae^{-Bx^2}$	$[0,+1]$	
Sinusoidal	$y = A\,\mathrm{sen}(\omega x + \varphi)$	$[-1,+1]$	

Deep Learning



Neuronal Networks

```
from sklearn.neural_network import MLPClassifier
X = [[0., 0.], [1., 1.]]
y = [0, 1]
clf = MLPClassifier(solver='lbfgs', alpha=1e-5, hidden_layer_sizes=(5, 2),
random_state=1)
clf.fit(X, y)
```

https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Neuronal Networks

clf.predict([[2., 2.], [-1., -2.]])

```
In [513]: from sklearn.neural_network import MLPClassifier
          X = [[0., 0.], [1., 1.]]
          y = [0, 1]
          clf = MLPClassifier(solver='lbfgs', alpha=1e-5,
                              hidden_layer_sizes=(5, 2), random_state=1)

          clf.fit(X, y)

Out[513]: MLPClassifier(activation='relu', alpha=1e-05, batch_size='auto', beta_1=0.9,
                        beta_2=0.999, early_stopping=False, epsilon=1e-08,
                        hidden_layer_sizes=(5, 2), learning_rate='constant',
                        learning_rate_init=0.001, max_fun=15000, max_iter=200,
                        momentum=0.9, n_iter_no_change=10, nesterovs_momentum=True,
                        power_t=0.5, random_state=1, shuffle=True, solver='lbfgs',
                        tol=0.0001, validation_fraction=0.1, verbose=False,
                        warm_start=False)

In [514]: clf.predict([[2., 2.], [-1., -2.]])

Out[514]: array([1, 0])
```

https://scikit-learn.org/stable/modules/neural_networks_supervised.html

MNIST

No es seguro | yann.lecun.com/exdb/mnist/

THE MNIST DATABASE

of handwritten digits

Yann LeCun, Courant Institute, NYU
Corinna Cortes, Google Labs, New York
Christopher J.C. Burges, Microsoft Research, Redmond

Please refrain from accessing these files from automated scripts with high frequency. Make copies!'

The MNIST database of handwritten digits, available from this page, has a training set of 60,000 examples, and a test set of 10,000 examples. It is a subset of a larger set available from NIST. The digits have been size-normaliz centered in a fixed-size image.

It is a good database for people who want to try learning techniques and pattern recognition methods on real-world data while spending minimal efforts on preprocessing and formatting.

Four files are available on this site:

```
train-images-idx3-ubyte.gz:  training set images (9912422 bytes)
train-labels-idx1-ubyte.gz:  training set labels (28881 bytes)
t10k-images-idx3-ubyte.gz:   test set images (1648877 bytes)
t10k-labels-idx1-ubyte.gz:   test set labels (4542 bytes)
```



http://yann.lecun.com/exdb/mnist/

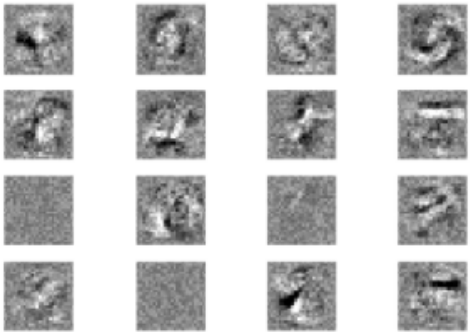
Visualización de pesos MNIST

```
import warnings
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.exceptions import ConvergenceWarning
from sklearn.neural_network import MLPClassifier
print(__doc__)
# Load data from https://www.openml.org/d/554
X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
X = X / 255.
# rescale the data, use the traditional train/test split
X_train, X_test = X[:60000], X[60000:]
y_train, y_test = y[:60000], y[60000:]

mlp = MLPClassifier(hidden_layer_sizes=(50,), max_iter=10, alpha=1e-4,
                    solver='sgd', verbose=10, random_state=1,
                    learning_rate_init=.1)
# this example won't converge because of CI's time constraints, so we catch the
# warning and are ignore it here
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=ConvergenceWarning,
                            module="sklearn")
    mlp.fit(X_train, y_train)
print("Training set score: %f" % mlp.score(X_train, y_train))
print("Test set score: %f" % mlp.score(X_test, y_test))
fig, axes = plt.subplots(4, 4)
# use global min / max to ensure all weights are shown on the same scale
vmin, vmax = mlp.coefs_[0].min(), mlp.coefs_[0].max()
for coef, ax in zip(mlp.coefs_[0].T, axes.ravel()):
    ax.matshow(coef.reshape(28, 28), cmap=plt.cm.gray, vmin=.5 * vmin,
               vmax=.5 * vmax)
    ax.set_xticks(())
    ax.set_yticks(())
plt.show()
```

Visualización de pesos MNIST

```
Automatically created module for IPython interactive environment
Iteration 1, loss = 0.32009978
Iteration 2, loss = 0.15347534
Iteration 3, loss = 0.11544755
Iteration 4, loss = 0.09279764
Iteration 5, loss = 0.07889367
Iteration 6, loss = 0.07170497
Iteration 7, loss = 0.06282111
Iteration 8, loss = 0.05530788
Iteration 9, loss = 0.04960484
Iteration 10, loss = 0.04645355
Training set score: 0.986800
Test set score: 0.970000
```



Neuronal Networks

Ventajas:

- Aprende modelos no lineales
- Aprendizaje en tiempo real (on-line learning)

Desventajas

- Problemas con mínimos locales
- Varios parámetros: número de neuronas, capas, iteraciones….
- Caja Negra (Black Box)

https://scikit-learn.org/stable/modules/neural_networks_supervised.html

References

https://ipython.org/

https://numpy.org/

https://jupyter.org/try

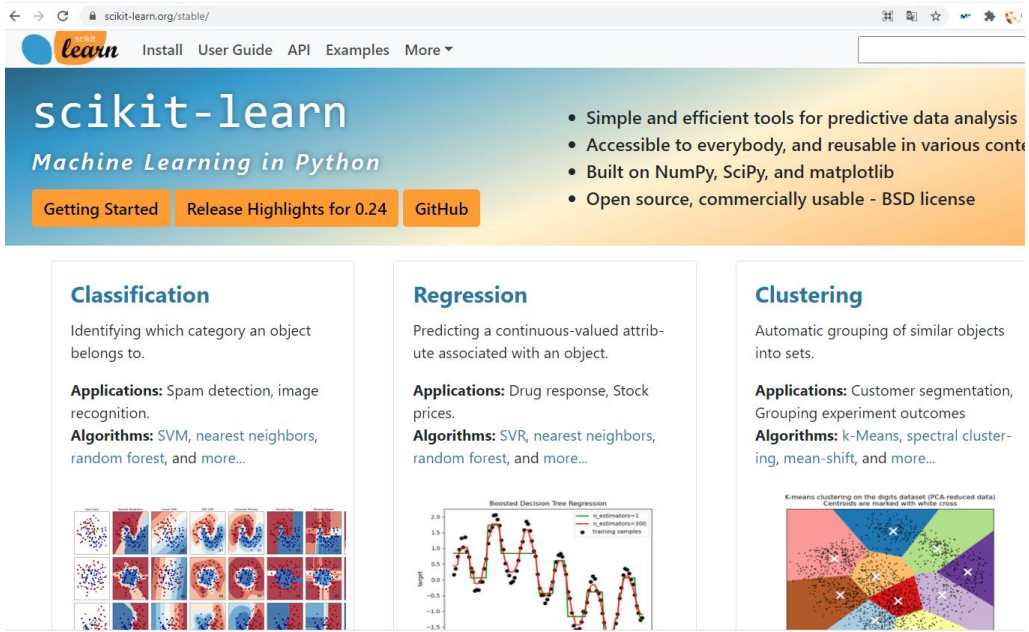
https://pandas.pydata.org/

http://seaborn.pydata.org/

https://scikit-learn.org/stable/

References

https://scikit-learn.org/stable/



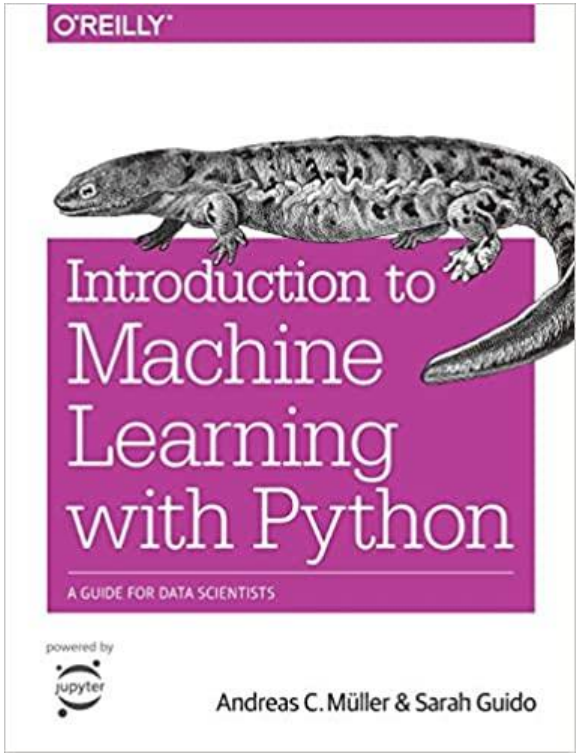
References

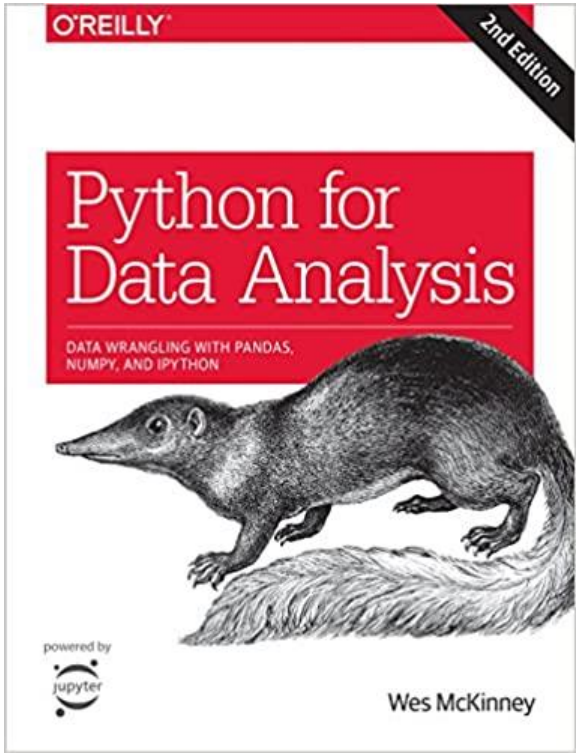
Face recognition example:

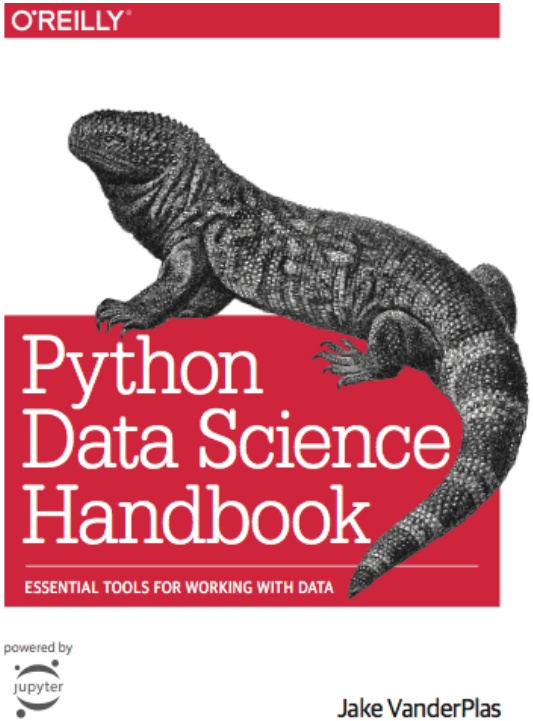
https://scikit-learn.org/stable/auto_examples/applications/plot_face_recognition.html#sphx-glr-auto-examples-applications-plot-face-recognition-py

https://jefflirion.github.io/udacity/Intro_to_Machine_Learning/Lesson12.html

Referencias







https://jakevdp.github.io/PythonDataScienceHandbook/

ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID

